

Dokumentation: caked_mask

Berechnung, Verarbeitung und Verwendung der Detektormaske
in der 2D-XRD Spannungsanalyse-GUI

1. Überblick

Die **caked_mask** beschreibt für jeden (chi, 2theta)-Bin im gecakten 2D-Integrationsbild, welcher Anteil der Detektorpixel in diesem Bin tatsächlich valide Messdaten enthält. Ein Wert von 1.0 bedeutet, dass alle Pixel in diesem Bin valide sind. Ein Wert von 0.0 bedeutet, dass keine Pixel vorhanden sind (außerhalb des Detektors oder vollständig maskiert).

Ergänzend wird die **valid_fraction** gespeichert — ein kontinuierliches Array mit Werten zwischen 0 und 1, das den echten Pixelanteil angibt. Im Gegensatz zur binären caked_mask ermöglicht valid_fraction die Erkennung auch partiell abgedeckter Bins (z.B. schmale Modulgrenzen).

2. Berechnung in Python (pyfai_multigeom_run.py)

2.1 Funktion compute_caked_mask()

Diese Funktion berechnet valid_fraction und caked_mask pixelbasiert. Sie wird einmal pro Alpha-Gruppe aufgerufen.

Schritt 1 — Detektormaske laden:

Die Pixelmaske wird aus dem ersten AzimuthalIntegrator (AI) gelesen. Maskierte Pixel haben den Wert True.

Schritt 2 — Synthetisches Bild erzeugen:

Pro Detektorbild wird ein synthetisches Bild erstellt: valide Pixel erhalten den Wert 1.0, maskierte Pixel den Wert 0.0.

Schritt 3 — Integration ohne Maske:

Das synthetische Bild (mit maskierten Pixeln = 0) und ein vollständig valides Referenzbild (alle Pixel = 1) werden beide mit integrate2d integriert. Die Division ergibt den echten Pixelanteil pro Bin:

```
valid_fraction = I_mask / I_full    (pro Bin)
```

Schritt 4 — Binaere Maske:

Aus valid_fraction wird eine binaere caked_mask berechnet. Ein Bin gilt als valide wenn der Pixelanteil ueber dem Schwellenwert von 0.001 liegt:

```
caked_mask = (valid_fraction > 0.001).astype(uint8)
```

Schritt 5 — Speichern:

Beide Arrays werden in einer separaten MAT-Datei gespeichert (Suffix _caked_mask.mat) und zusaetzlich in die Haupt-MAT-Datei der Alpha-Gruppe eingefuegt.

2.2 Ausgabedateien

Datei	Inhalt
-------	--------

*_alpha{X}.mat	Haupt-Integration + caked_mask + valid_fraction
*_alpha{X}_caked_mask.mat	caked_mask, valid_fraction, chi_valid_caked
*_alpha{X}_caked_mask.npz	Gleiche Daten im NumPy-Format

3. Verarbeitung in MATLAB

3.1 opengammafilecallback()

Nach der pyFAI-Integration wird pro Alpha-Gruppe die zugehoerige caked_mask geladen und in pyfaiOutPerAlpha{k} gespeichert. Es gilt folgende Prioritaet:

- 1. Alpha-spezifische _caked_mask.mat Datei (aus separater Berechnung)
- 2. Fallback: kombinierte caked_mask aus der Gesamt-Integration

Die relevanten Felder in pyfaiOutPerAlpha{k} nach dem Laden:

```
pyfaiOutPerAlpha{k}.caked_mask [npt_azim x npt_rad] uint8
pyfaiOutPerAlpha{k}.valid_fraction [npt_azim x npt_rad] float32
```

3.2 runBinning()

Beim Binning wird valid_fraction verwendet um chi-Bins zu identifizieren, die durch Detektorluecken beeintraehtigt sind. Der Parameter ratioThreshold (Standard: 0.05) filtert Bins mit zu geringem Intensitaetsverhaeltnis zum Median.

3.3 markInvalidGammaRegions()

Diese Funktion markiert im eps(gamma)-Plot zwei Arten von ungueltigem Bereich:

- **Grau:** Bins die durch BinnedGammaValid als unguelig markiert sind (aus chi_valid und ratioThreshold)
- **Orange:** Bins bei denen valid_fraction am Peak-2theta-Wert unter peakMaskThresh faellt

Fuer die orangenen Markierungen wird fuer jeden gamma-Bin der mittlere valid_fraction-Wert in einem Fenster von peakTthWin Grad um die Peak-2theta-Position berechnet:

```
peakMaskFrac(bn) = mean(valid_fraction(idxChi, idxTthPk))
```

Hinweis: Die Funktion laedt valid_fraction einmalig vor der Schleife, um Performance zu optimieren. Peak-2theta wird bevorzugt aus Spalte 9 von FitDataMod gelesen (gefittete Werte).

3.4 applyDetectorMaskToFitData()

Setzt Fitdaten-Zeilen in FitDataMod auf NaN fuer alle gamma-Bins, bei denen peakMaskFrac unter peakMaskThresh liegt. Verwendet ebenfalls valid_fraction statt caked_mask.

3.5 SliderCallbackPlotRawData() — Plot intensity data Tab

Im Intensity-Plot werden Detektorluecken als graue Patches visualisiert. Es wird valid_fraction des aktuellen gamma-Bins verwendet, gemittelt ueber halfWin Nachbarbins in chi-Richtung.

3.6 showCakedMaskOverBins()

Visualisierungsfunktion die caked Image, valid_fraction-Profil und Overlay in einer Figure zeigt. Verwendet valid_fraction direkt fuer alle Darstellungen. Wird manuell aufgerufen:

```
h = guidata(gcf);  
showCakedMaskOverBins(h, 1); % Alpha-Gruppe 1
```

4. Einflussparameter

Parameter	Ort	Standard	Wirkung
peakMaskThresh	GUI EditField peakMaskThreshEdit	0.99	Schwellenwert fuer valid_fraction. Bins mit frac < thresh werden orange markiert und koennen geloescht werden. 0.99 = schon 1% maskierte Pixel fuehren zur Markierung.
peakTthWin	GUI EditField peakTthWinEdit	0.5	2theta-Fenster in Grad um die Peak-Position fuer die Masken- prufung. Groesseres Fenster mittelt mehr 2theta-Kanaele.
halfWin (trackChiAvgBinsEdit)	GUI EditField	4	Anzahl Nachbarbins in chi-Richtung fuer die Mittelung von valid_fraction. Groesserer Wert = robuster aber niedrigere chi-Aufoesung.
ratioThreshold (in runBinning)	Hardcodiert	0.05	Minimum-Intensitaetsverhaeltnis (Minimum/Median) fuer BinnedGamma- Valid. Bins unter diesem Wert werden grau markiert.
npt_rad	cfg in MATLAB / job.json	3000	Anzahl 2theta-Kanaele der Integration. Hoehere Werte = feinere 2theta- Aufoesung der valid_fraction.
npt_azim	cfg in MATLAB / job.json	360	Anzahl chi-Kanaele. Hoehere Werte ermoeglichen feinere Erkennung von Detektorluecken in chi.

5. Datenfluss — Zusammenfassung

Schritt	Funktion / Datei	Ergebnis
1	compute_caked_mask() pyfai_multigeom_run.py	valid_fraction [npt_azim x npt_rad] caked_mask [npt_azim x npt_rad]
2	opengammafilecallback() XRD2DStressAnalysis_modPV_pyFAI.m	pyfaiOutPerAlpha{k}.valid_fraction pyfaiOutPerAlpha{k}.caked_mask
3	runBinning()	BinnedGammaValid{k} — logischer Vektor BinnedGammaRaw{k} — chi-Werte
4	fitpeakscallback()	FitDataMod{k} — Fitdaten pro Bin (Spalte 9: gefittete 2theta-Werte)
5	markInvalidGammaRegions()	Graue/orange Patches im eps(gamma)-Plot

6	<code>applyDetectorMaskToFitData()</code>	FitDataMod{k}: betroffene Zeilen = NaN
7	<code>SliderCallbackPlotRawData()</code>	Graue Patches im Intensity-Plot
8	<code>showCakedMaskOverBins()</code>	Visualisierung: Caked Image + Masken-Overlay + frac_valid-Profile

6. Typische Arbeitsabfolge

- **1. PONI-Dateien laden** — `opengammafilecallback()` berechnet `valid_fraction` und speichert sie in `pyfaiOutPerAlpha`.
- **2. Caked Image pruefen** — `showCakedMaskOverBins(h, 1)` zeigt das Overlay. Der Slider ermoeeglicht die Kontrolle jedes gamma-Bins.
- **3. Peaks fitten** — `fitpeakscallback()` erstellt `FitDataMod` mit gefitteten 2theta-Werten in Spalte 9.
- **4. Masken-Markierungen pruefen** — `markInvalidGammaRegions()` zeigt graue und orange Patches im `eps(gamma)`-Plot. `peakMaskThresh` im `EditField` anpassen.
- **5. Detektormaske anwenden** — `applyDetectorMaskToFitData()` setzt betroffene Bins auf NaN.
- **6. Stressfit berechnen** — `fitstressdatacallback()` verwendet nur valide Bins fuer die Spannungsberechnung.
- **7. Manuelle Korrektur** — `modstressdatacallback()` erlaubt das manuelle Loeschen verbliebener Ausreisser per Lasso.

Hinweis: `peakMaskThresh = 0.99` ist der empfohlene Wert. Er markiert jeden Bin bei dem auch nur 1% der Detektorpixel am Peak maskiert sind. Bei sehr schmalen Modulgrenzen (1-2 Pixel) kann ein niedrigerer Wert wie 0.95 sinnvoll sein um Fehlmarkierungen zu vermeiden.